

Encapsulation in Object-Oriented Programming

Zadid Bin Azad

Department of Computer Science and Engineering

Islamic University of Technology

zadidbinazad@iut-dhaka.edu

May 14, 2026

Abstract

This document is a structured teaching guide for introducing the concept of *encapsulation* in object-oriented programming. Rather than presenting encapsulation as a mechanical exercise of attaching access modifiers to fields, the guide treats it as a design philosophy: an object should manage its own state and expose only what callers genuinely need. The material is organised around a single running scenario — a banking system in which a `BankAccount` class is built up incrementally. At each stage of the development, the instructor poses a design question to the class, highlights the most common student mistake, and demonstrates how the encapsulation principle resolves it. The intended audience is undergraduate students who have prior exposure to procedural programming but are encountering object-oriented design for the first time.

1 Introduction

Encapsulation is one of the four canonical pillars of object-oriented programming, alongside abstraction, inheritance, and polymorphism. However, in introductory courses, encapsulation is frequently reduced to a syntactic ritual: students are told to mark fields as `private` and provide a `get/set` pair for each one. This presentation hides the actual purpose of the principle and produces classes that are technically “encapsulated” yet offer no real protection of state.

The position taken in this guide is that encapsulation is a *contract*. An object promises to keep itself in a valid, internally consistent state at all times. To honour that promise, the object must:

1. bundle the data and the operations that act upon it into a single unit;
2. hide the data that the outside world has no reason to touch;
3. refuse to be constructed in an invalid state;
4. validate every change before accepting it; and
5. compute, rather than store, anything that can be derived from existing state.

The remainder of this guide develops each of these obligations through a single scenario.

Teaching Insight

Encapsulation is best taught *after* students have felt the pain of unprotected data. Begin with a deliberately bad design, let the class break it, and only then introduce the corrective principle. The lesson lands harder when students discover the problem themselves.

2 The Running Scenario: A Banking System

Consider a small retail bank that needs a software model for a customer's account. The bank's real-world business rules include the following:

- every account has a unique account number assigned by the bank, never by the customer;
- the balance of any account must never be negative;
- every account must have an owner and a currency at the moment it is opened;
- deposits and withdrawals must each be a positive amount;
- the bank wishes to know, on demand, how long an account has been open and whether it qualifies as a “premium” account;
- internal validation rules (such as transaction limits) are bank policy and must not be reachable or modifiable by client code.

These rules will reappear, one by one, as the class is developed. Each rule will force a design decision, and each decision will reveal a different facet of encapsulation.

3 Starting from a Procedural Mindset: The Naive Record

A student arriving from a procedural background will typically begin by treating a class as a passive bag of fields, much like a C `struct`.

```
1 public class BankAccount {  
2     public String accountNumber;  
3     public String ownerName;  
4     public double balance;  
5     public String currency;  
6 }
```

Listing 1: A first attempt: data only, no protection.

This compiles. It even appears to work. The instructor should now stress-test it in front of the class.

Question for the Class

If every field is publicly accessible, what prevents an external piece of code from writing `account.balance = -50000`; or `account.accountNumber = ""`? Whose responsibility is it to ensure the account remains valid?

Where Students Typically Slip

Students often respond that “the programmer will simply remember not to do that” or that “we can write a separate function to check correctness.” Both answers misplace the responsibility: correctness becomes a property of every caller, rather than of the object itself.

How Encapsulation Resolves It

The object must *own* its correctness. The class itself, not its callers, must guarantee that no `BankAccount` instance ever holds an invalid balance, an empty owner name, or an unset currency. This is the first commitment of encapsulation.

4 Bundling Data with Behaviour

Once students accept that the class must defend itself, the next step is to give it the means to do so: the operations that act on the data should live inside the class.

Question for the Class

Imagine that the functions `deposit()` and `withdraw()` are written as free-standing functions that take a `BankAccount` as a parameter. What goes wrong when a new programmer joins the team and writes their own `transferMoney()` function elsewhere?

Where Students Typically Slip

Students often see no harm in scattering behaviour across many files: “as long as the function is correct, it does not matter where it lives.” What they miss is that there is now no single place that defines what an account is allowed to do, and no way to enforce that every transaction goes through the same validation path.

How Encapsulation Resolves It

By placing `deposit()` and `withdraw()` inside the class, the object becomes the sole gate-keeper of its own state changes. This is the principle of *cohesion*: behaviour lives next to the data it manipulates, and every change to that data passes through a known door.

```
1 public class BankAccount {
2     public String accountNumber;
3     public String ownerName;
4     public double balance;
5     public String currency;
6
7     public void deposit(double amount) {
8         balance = balance + amount;
9     }
10
11    public void withdraw(double amount) {
12        balance = balance - amount;
13    }
14 }
```

Listing 2: Operations begin to migrate inside the class.

The class now exhibits cohesion, but it remains defenceless: the fields are still exposed, and `withdraw` will happily produce a negative balance. The next section closes that gap.

5 Hiding Internal State

The most visible mechanism of encapsulation is the access modifier. Students learn the keyword `private` early, but rarely understand which fields deserve it and which do not.

Question for the Class

Look at the four fields of `BankAccount`. Which ones, if any, should the outside world be able to modify directly? Which should it be able to read but not modify? Which should it not be able to see at all?

Where Students Typically Slip

Two extremes appear in student work. Some students mark every field `private` but then add a public setter for each, which restores the original problem with extra ceremony. Others leave fields public “for convenience during testing” and forget to revert the change.

How Encapsulation Resolves It

The decision is not mechanical. A field should be `private` if its value participates in any invariant of the class, or if its representation might plausibly change in the future. For our account, *all four* fields meet that test: the balance has an invariant, the account number is bank-assigned, the owner cannot be silently swapped, and the currency is fixed at opening.

```
1 public class BankAccount {  
2     private String accountNumber;  
3     private String ownerName;  
4     private double balance;  
5     private String currency;  
6  
7     public void deposit(double amount) {  
8         balance = balance + amount;  
9     }  
10  
11    public void withdraw(double amount) {  
12        balance = balance - amount;  
13    }  
14 }
```

Listing 3: Fields hidden behind the private modifier.

The compiler now refuses any external assignment to a field. But this introduces a new question: how does an account ever come to exist in the first place?

6 Controlled Creation Through Constructors

Once the fields are private, the default no-argument construction will produce an account with a null owner, a null currency, and a zero balance. That is a broken object: it satisfies the type system but violates the bank's rules.

Question for the Class

If a `BankAccount` object can come into existence with no owner and no currency, at what point does the object become valid? Whose job is it to fill in the missing fields, and what happens if they forget?

Where Students Typically Slip

Students often plan to “call `setOwner()` right after creation,” which leaves a window in which an invalid object exists. They also tend to overlook the fact that an object referenced from another thread, or passed to another method, may be observed during that window.

How Encapsulation Resolves It

A constructor is the only point at which an object's existence is negotiated. It must therefore demand exactly the information required to produce a valid object, and it must reject any attempt to construct an invalid one. There is no such thing as a half-built `BankAccount`.

```
1 public BankAccount(String accountNumber, String ownerName, String currency) {  
2     if (accountNumber == null || accountNumber.isBlank()) {  
3         throw new IllegalArgumentException("Account number is required.");  
4     }  
5     if (ownerName == null || ownerName.isBlank()) {  
6         throw new IllegalArgumentException("Owner name is required.");  
7     }  
8     if (currency == null || currency.isBlank()) {  
9         throw new IllegalArgumentException("Currency is required.");  
10    }  
11    this.accountNumber = accountNumber;  
12    this.ownerName = ownerName;  
13    this.currency = currency;  
14    this.balance = 0.0;  
15 }
```

Listing 4: A constructor that refuses invalid arguments.

7 Constructor Overloading: Many Valid Births

Real-world banks open accounts in several distinct ways: a brand-new account with no opening deposit, an account opened with a cash deposit, and an account migrated from a partner bank with an existing balance. Each is a legitimate but distinct construction story.

Question for the Class

If we want to support multiple opening scenarios, should we add optional parameters, write factory functions, or provide several constructors? What does each choice cost?

Where Students Typically Slip

A frequent error is to write a single constructor that accepts every possible parameter and let some of them default to `null` or zero. This pushes the validation problem back out to the caller, who must now remember which combinations are legal.

How Encapsulation Resolves It

Constructor overloading offers a fixed, finite set of valid creation paths, each of which produces a fully initialised object. The class declares, in code, exactly how it may be brought into existence; the compiler enforces the rest.

```
1 public BankAccount(String accountNumber, String ownerName, String currency) {  
2     this(accountNumber, ownerName, currency, 0.0);  
3 }  
4  
5 public BankAccount(String accountNumber, String ownerName,  
6                     String currency, double openingDeposit) {  
7     if (accountNumber == null || accountNumber.isBlank())  
8         throw new IllegalArgumentException("Account number is required.");  
9     if (ownerName == null || ownerName.isBlank())
```

```

10         throw new IllegalArgumentException("Owner name is required.");
11     if (currency == null || currency.isBlank())
12         throw new IllegalArgumentException("Currency is required.");
13     if (openingDeposit < 0.0)
14         throw new IllegalArgumentException("Opening deposit cannot be negative.
15         ");
16
17     this.accountNumber = accountNumber;
18     this.ownerName      = ownerName;
19     this.currency       = currency;
20     this.balance        = openingDeposit;
21     this.openedOn       = LocalDate.now();
22 }
23
24 public BankAccount(BankAccount migratedFromPartnerBank) {
25     this(migratedFromPartnerBank.accountNumber,
26         migratedFromPartnerBank.ownerName,
27         migratedFromPartnerBank.currency,
28         migratedFromPartnerBank.balance);
29 }

```

Listing 5: Three legitimate ways to open an account.

Teaching Insight

Emphasise that each overload *ends* with a fully valid object. Overloading is not about saving keystrokes; it is about acknowledging that there are several distinct, equally legitimate stories by which an object can come into being.

8 Controlled Access: Getters and Validating Setters

Once construction is safe, attention turns to the object's lifetime. Some attributes must be readable; some must be modifiable, but only under tightly controlled circumstances; some must be neither.

Question for the Class

Should the `balance` field have a setter? If not, how does the balance ever change? If yes, what should that setter check before accepting a new value?

Where Students Typically Slip

The IDE-generated “getter and setter for every field” is the textbook anti-pattern. A blind `setBalance(double newBalance)` undoes every protection the class has built up, because the caller can pass any number, including a negative one, and bypass `deposit` and `withdraw` entirely.

How Encapsulation Resolves It

A field deserves a setter only when external modification is part of the class's intended behaviour, and even then the setter must validate. The balance, in our case, has no setter at all: it changes only through `deposit` and `withdraw`, each of which enforces its own preconditions.

```

1 public double getBalance() {
2     return balance;

```

```
3 }
4
5 public String getAccountNumber() {
6     return accountNumber;
7 }
8
9 public String getOwnerName() {
10    return ownerName;
11 }
12
13 public void deposit(double amount) {
14    if (amount <= 0.0)
15        throw new IllegalArgumentException("Deposit must be positive.");
16    balance = balance + amount;
17 }
18
19 public void withdraw(double amount) {
20    if (amount <= 0.0)
21        throw new IllegalArgumentException("Withdrawal must be positive.");
22    if (amount > balance)
23        throw new IllegalStateException("Insufficient funds.");
24    balance = balance - amount;
25 }
```

Listing 6: Read-only access for the balance, validated mutation through dedicated operations.

9 Computed Properties: State That Should Not Be Stored

The bank wishes to know how long an account has been open and whether it qualifies as a premium account. A student's first instinct is to add fields for these.

Question for the Class

If we store `ageInDays` as a field, when must it be updated? What happens if a method forgets to update it? Is there a simpler design that cannot go wrong?

Where Students Typically Slip

Storing derived values is one of the most common sources of silent bugs. Two fields that should be related (such as `balance` and `isPremium`) can drift out of sync, and the object will report contradictory information depending on which getter is called.

How Encapsulation Resolves It

Anything that can be computed from existing state should be computed on demand. The class keeps a single source of truth (the date the account was opened, the current balance) and derives everything else inside read-only methods. The class is the only piece of code that knows the derivation rule.

```
1 public long getAgeInDays() {
2     return ChronoUnit.DAYS.between(openedOn, LocalDate.now());
3 }
4
5 public boolean isPremium() {
6     return balance >= PREMIUM_THRESHOLD && getAgeInDays() >= 365;
7 }
```

Listing 7: Derived properties are computed, never stored.

10 Private Helper Methods: The Hidden Machinery

As the class grows, validation logic and bookkeeping operations begin to repeat. The temptation is to extract them into helper methods. The next question is whether those helpers are part of the class's public face.

Question for the Class

A method `validateAmount(double)` is useful inside both `deposit` and `withdraw`. Should it be public so that other classes can also call it? What is the cost of saying “yes”?

Where Students Typically Slip

Students often make every helper `public` in case “somebody else might need it later.” The cost is that every public method becomes part of the class's contract: once external code depends on it, it cannot be removed or renamed without breaking that code.

How Encapsulation Resolves It

The public interface of a class is a promise to the rest of the system. Promises are expensive: each one constrains future change. A method that exists purely to support the class's own operations should be `private`, so that it can evolve freely.

```
1 private void requirePositive(double amount, String label) {
2     if (amount <= 0.0)
3         throw new IllegalArgumentException(label + " must be positive.");
4 }
5
6 public void deposit(double amount) {
7     requirePositive(amount, "Deposit");
8     balance = balance + amount;
9 }
10
11 public void withdraw(double amount) {
12     requirePositive(amount, "Withdrawal");
13     if (amount > balance)
14         throw new IllegalStateException("Insufficient funds.");
15     balance = balance - amount;
16 }
```

Listing 8: A private helper used only inside the class.

11 Class Invariants: The Rules the Object Enforces

With every previous section in place, the instructor can now name the property that ties them all together: the *class invariant*. An invariant is a statement that must be true for every instance of the class, at every observable moment of its lifetime.

Question for the Class

What statements should always be true of a **BankAccount** instance, regardless of what has happened to it? Write them down before looking at the next paragraph.

For our account, a reasonable set of invariants includes:

- the account number is non-null and non-blank;
- the owner name is non-null and non-blank;
- the currency is non-null and non-blank;
- the balance is greater than or equal to zero;
- the opening date is in the past or present, never the future.

Where Students Typically Slip

A subtle error is to enforce invariants only in the constructor and forget about them in subsequent methods. Students reason “if the object started valid, it must still be valid,” which is only true if every state-changing method also checks its own preconditions.

How Encapsulation Resolves It

Every public method that can change state must, on entry, verify its preconditions and, on exit, leave the object in a state where every invariant still holds. Encapsulation is the umbrella mechanism that makes this enforcement possible: because no outsider can touch the fields, the class is the only place such enforcement needs to live.

Teaching Insight

A useful classroom exercise is to ask each student to write down one invariant and then describe the smallest possible change to the class that would break it. This exposes how fragile state becomes when fields are accessible from outside.

12 The Fully Encapsulated Class

The synthesis below collects every decision made so far into a single class. The instructor is encouraged to develop it on the board, section by section, in the order of the preceding chapters.

```
1 import java.time.LocalDate;
2 import java.time.temporal.ChronoUnit;
3
4 public class BankAccount {
5
6     private static final double PREMIUM_THRESHOLD = 100_000.0;
7
8     private final String accountNumber;
9     private final String currency;
10    private final LocalDate openedOn;
11
12    private String ownerName;
13    private double balance;
14
15    public BankAccount(String accountNumber, String ownerName, String currency)
16    {
```

```
16         this(accountNumber, ownerName, currency, 0.0);
17     }
18
19     public BankAccount(String accountNumber, String ownerName,
20                        String currency, double openingDeposit) {
21         if (accountNumber == null || accountNumber.isBlank())
22             throw new IllegalArgumentException("Account number is required.");
23         if (ownerName == null || ownerName.isBlank())
24             throw new IllegalArgumentException("Owner name is required.");
25         if (currency == null || currency.isBlank())
26             throw new IllegalArgumentException("Currency is required.");
27         if (openingDeposit < 0.0)
28             throw new IllegalArgumentException("Opening deposit cannot be
29                 negative.");
30
31         this.accountNumber = accountNumber;
32         this.ownerName     = ownerName;
33         this.currency      = currency;
34         this.balance       = openingDeposit;
35         this.openedOn      = LocalDate.now();
36     }
37
38     public BankAccount(BankAccount migratedFromPartnerBank) {
39         this(migratedFromPartnerBank.accountNumber,
40             migratedFromPartnerBank.ownerName,
41             migratedFromPartnerBank.currency,
42             migratedFromPartnerBank.balance);
43     }
44
45     public String getAccountNumber() { return accountNumber; }
46     public String getOwnerName()     { return ownerName; }
47     public String getCurrency()      { return currency; }
48     public double getBalance()        { return balance; }
49     public LocalDate getOpenedOn()    { return openedOn; }
50
51     public long getAgeInDays() {
52         return ChronoUnit.DAYS.between(openedOn, LocalDate.now());
53     }
54
55     public boolean isPremium() {
56         return balance >= PREMIUM_THRESHOLD && getAgeInDays() >= 365;
57     }
58
59     public void deposit(double amount) {
60         requirePositive(amount, "Deposit");
61         balance = balance + amount;
62     }
63
64     public void withdraw(double amount) {
65         requirePositive(amount, "Withdrawal");
66         if (amount > balance)
67             throw new IllegalStateException("Insufficient funds.");
68         balance = balance - amount;
69     }
70
71     public void renameOwner(String newOwnerName) {
72         if (newOwnerName == null || newOwnerName.isBlank())
73             throw new IllegalArgumentException("Owner name is required.");
74         this.ownerName = newOwnerName;
75     }
76
77     private void requirePositive(double amount, String label) {
```

```
77         if (amount <= 0.0)
78             throw new IllegalArgumentException(label + " must be positive.");
79     }
80 }
```

Listing 9: The complete `BankAccount` class.

Several design choices in the final form deserve explicit attention in class. The fields `accountNumber`, `currency`, and `openedOn` are declared `final`: they are part of the account's identity and must never change after construction. The owner name, by contrast, is mutable, but only through the dedicated method `renameOwner`, which validates its input. The balance has no setter at all; it changes only as a side-effect of a deposit or a withdrawal. Every public entry point either rejects bad input outright or preserves the invariants of the class.

13 Synthesis and Classroom Discussion

By the end of the development, the class should have internalised six related observations.

1. Encapsulation is a contract between the object and the rest of the world. The keyword `private` is one of its tools, but not its substance.
2. An object that depends on its callers to keep it valid is not encapsulated, regardless of how its fields are declared.
3. A constructor is the only opportunity to refuse the creation of an invalid object; overloading allows the class to acknowledge several legitimate creation stories.
4. Setters are not free. Each one widens the surface through which the object's invariants can be attacked.
5. Derived state should be computed, not stored, so that the object cannot contradict itself.
6. The public interface is a long-term commitment; helpers that exist for the class's own convenience belong on the private side of the boundary.

A productive closing exercise is to hand students a domain that is structurally similar but unfamiliar — a library book, a hospital admission, an examination grade — and ask them to redesign it from scratch, articulating the invariants, the construction stories, and the public interface before writing a single line of code. The goal is not to produce code that compiles, but to produce code whose objects can defend themselves.